

FastAPI 개발환경 준비: conda 또는 venv

conda

1. miniconda 다운로드 및 설치, Anaconda Prompt 실행 후
2. `conda create -n fastapi_env python=3.12`
3. `conda activate fastapi_env`
4. `python -m pip install ipykernel`
5. `python -m ipykernel install --user --name fastapi_env --display-name "FastAPI kernel"`
→ 현재 활성화된 파이썬 환경(fastapi_env)을 Jupyter가 인식할 수 있는 커널로 등록하는 명령어

- `--user`: 시스템 전체가 아니라 현재 사용자 계정에만 등록
- `--name fastapi_env`: 내부적으로 식별할 커널의 이름
- `--display-name "FastAPI kernel"`: 사용자에게 보여지는 이름

python이 실행되는 현재 가상환경을 이름 "fastapi_env"로 등록, 화면 표시는 "--display-name"로 지정
conda와 Jupyter의 이름 체계는 별개임

6. `python -m pip install fastapi uvicorn`
7. `conda init powershell`
→ PowerShell은 시작할 때마다 프로필(profile) 스크립트를 읽어서 설정 관련 작업을 실행
→ `conda init powershell`은 프로필 파일에 conda 관련 초기화 코드를 작성
→ 변경 사항이 적용되려면 powershell을 다시 시작해야 함
※ command prompt에서 사용하려면 `conda init cmd.exe`
8. `conda config --set auto_activate_base false`
→ PowerShell 시작 시, 기본 conda 환경 base를 자동으로 활성화하는데 이 설정을 끄는 명령
※ 이 설정은 전역 설정이므로 powershell/command prompt 둘 중의 한 군데서만 하면 됨

※ 단계 7, 8이 필요한 이유

- 우리가 필요한 것은 uvicorn 프로그램임, uvicorn이 있어야 웹 서버가 돌아감
- uvicorn은 conda 가상환경에 설치되어 있으므로, 그 가상환경을 활성화 해야 함
- uvicorn은 셸에서 실행해야 하는데, 셸 중에서 powershell을 쓴다고 가정하면
- powershell에서 conda 명령어가 사용이 가능해야 함 (활성화를 해야 하므로) → 7번 단계
- 이 시점에서 powershell을 실행하면 auto_activate_base 설정의 기본값은 true이므로 conda base 가상환경이 자동으로 활성화 됨 → 8번 단계
- 나중에 노트북에서 커널을 선택하거나, .py 파일에서 인터프리터를 선택하면, VS Code가 그 가상환경의 파이썬 실행 파일을 직접 지정해서 코드를 실행하기 때문에, conda activate 필요없이 사용가능

VS Code에서

9. Python, Pylance, Jupyter extension 설치

A. VS Code에서 커널 목록에서 "FastAPI kernel" 선택

B. Ctrl+Shift+P → "Terminal: Select Default Profile" → "Windows Powershell" 선택

→ 어떤 셸을 사용하지 지정

→ 위에서 한 내용은 powershell 사용을 전제로 한 것이기 때문

→ 이후 "New Terminal" 명령에 적용

C. 메뉴 Terminal → New Terminal

D. conda activate fastapi_env 실행

★ VS Code는 현재 선택된 인터프리터/커널에 따라 해당 가상 환경을 자동으로 로딩함

★ 따라서 D는 필요 없을 수 있음

E. uvicorn --version 실행해서 설치 확인

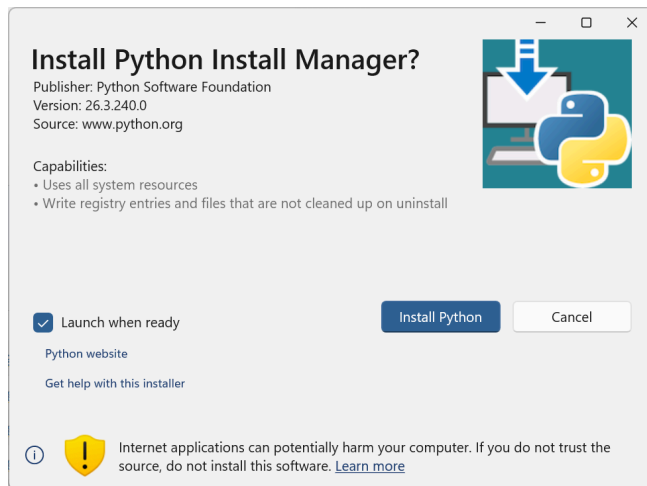
venv

1. <https://www.python.org/downloads/>

※ This is a security release of Python 3.xx → 다운로드 대상이 아님, Python 3.13 버전 이상 선택

2. Download 클릭 → windows (Download Python install manager) → windows (Download installer (MSIX))

※ "Launch when ready" 선택하지 않고 실행

[illegible]

```
Windows PowerShell
PS E:\아이티고동영상\FastAPI> .\fastapi_venv\Scripts\Activate.ps1
(fastapi_venv) PS E:\아이티고동영상\FastAPI> |
```

3. 프로젝트 폴더로 이동 (e.g., E:\아이티고동영상\FastAPI)

4. `python -m venv fastapi_venv`

→ 이름 `fastapi_venv`으로 가상환경(virtual environment)을 새로 생성

5. `dir` 실행해서 `fastapi_venv` 폴더 확인

6. `.\fastapi_venv\Scripts\Activate.ps1`

→ 가상환경 `fastapi_venv`을 활성화

※ `venv`는 시스템에 설치된 파이썬을 사용하고 `pip install`로 설치하는 라이브러리만으로 가상환경을 구성

※ `venv` 가상환경의 격리 대상은 라이브러리, `conda` 가상환경의 격리 대상은 파이썬 + 라이브러리

7. `python -m pip install ipykernel`

8. `python -m ipykernel install --user --name fastapi_venv --display-name "FastAPI venv kernel"`

→ 현재 활성화된 파이썬 환경(`fastapi_venv`)을 Jupyter가 인식할 수 있는 커널로 등록하는 명령어

- `--user`: 시스템 전체가 아니라 현재 사용자 계정에만 등록
- `--name fastapi_venv`: 내부적으로 식별할 커널의 이름
- `--display-name "FastAPI venv kernel"`: 사용자에게 보여지는 이름

9. `python -m pip install fastapi uvicorn`

A. VS Code에서 Python, Pylance, Jupyter extension 설치 - 이미 실행

B. VS Code에서 "FastAPI venv kernel" 선택

㉠ 결과적으로 ``fastapi_env``와 ``fastapi_venv``이 만들어짐

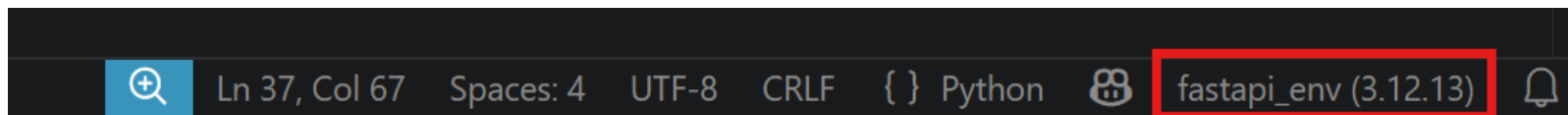
㉡ 차이는 `fastapi_env`는 conda, `fastapi_venv`는 venv이며 저장 위치가 다름

예를 들어,

`fastapi_env`는 `"C:\Users\jongy\miniconda3\envs\fastapi_env"`에, `fastapi_venv`는 프로젝트 폴더인 `"E:\아이티고동영상\FastAPI\fastapi_venv"`에 저장

㉢ ``fastapi_env``와 ``fastapi_venv``는 두 가지 용도로 사용

- VS Code에서 커널로 선택해서 주피터 노트북을 실행
- `.py` 파일의 실행에 사용하는 파이썬 인터프리터 역할



VS Code에서 인터프리터 선택은 화면 우측 하단의, 위 그림의 빨간색 표시 부분 클릭해서 선택